双语双部署架构方案 v1.0

双语双部署架构方案（CN / EN 双系统）

> 版本 v1.0 ｜ 2026-10-25 ｜ 决策：一仓·一 schema·两部署·分库·参考表加 `_en` 列 上游讨论：产品最终形态为**两套系统**（中文版 + 英文版），服务器位于**不同国家**； 英文版面向入境游客（Inbound），即《规划引擎升级》§14 的「入境视角」。 本文档为该决策的定稿设计与实施依据，后续会话以本文为准。

---

1. 背景与决策问题

事实	影响
两套系统：中文版（CN）+ 英文版（EN），面向不同用户群	UI 文案、数据呈现语言不同
服务器在不同国家	数据合规（CN 数据留 CN / ICP）、延迟、供应商可达性都按区域拆
CN 侧供应商（高德/12306/携程/智谱）为语料与能力底座	英文版的参考数据**只能**在 CN 侧生产
同一实体（同一个博物馆/餐厅/车次）两种语言	翻译是**同一行的属性**，不是另一张表

决策问题：两套系统的代码与数据如何组织—— **A. 两个项目两个仓库**；**B. 一个项目 + `_en` 后缀镜像表**；**C. 同表 + `_en` 平行列。**

2. 结论（定稿）

一个仓库、一条 schema（alembic 链）、两处部署（region=CN / region=INTL）、每区域独立数据库；共享实体表加 `_en` 平行列，用户数据按区域分库永不同步，参考数据 CN→海外单向同步。

即方案 **C**，叠加「分库双部署」。H5 界面文案走 i18n 字典，不进数据库。

3. 备选方案对比

方案	结论	理由
A. 两仓两项目	❌	solver/geo/营业时间/免签/覆盖率/天气等 95% 逻辑共享；每个 bugfix 修两遍，必然分叉漂化
B. `pois_en` 镜像表	❌	同一实体两张表=同步噩梦：`aggregate_poi_stats` 夜作业、`refresh_pois`、各 backfill 全部双写；漏一次写，两系统对同一场馆口径不一致

方案	结论	理由
C. 同表 + `_en` 列	✅	译文是行的属性；评分/营业时间/坐标/标签单一事实源，永不漂化；一次迁移 + 一次翻译 backfill 即生效

4. 目标架构

```
┌──────────── CN 区域（region=CN）────────────┐
│ H5（zh-CN） FastAPI(.env: CN) PG-CN          │
│ 高德/12306/携程/智谱 直连（语料生产侧）       │
│ 用户/行程/反馈数据只留 PG-CN，永不出境        │
└──────────────┬──────────────────────────────┘
               │ 参考表导出 → 单向同步（幂等 upsert，每日）
               ▼
┌──────────── 海外区域（region=INTL）─────────┐
│ H5（en） FastAPI(.env: INTL) PG-INTL         │
│ GLM 国际端点或本区 LLM；SMTP 本区            │
│ 高德经 CN 代理回源（config 预留注释）        │
│ 用户/行程/反馈数据只留 PG-INTL 不回流        │
│ 参考表导入作业（只读消费，不回写）           │
└──────────────────────────────────────────────┘
```

要点：

- **代码同一份**：同一镜像两次部署，仅 `.env` 不同（`region` 等开关见 §7）。
- **语料单向流**：参考数据只在 CN 生产（高德/携程/12306 仅 CN 可达），导出→导入同步到海外；海外侧只读消费。
- **用户数据双向不流动**：合规（数据驻留）+ 延迟（本区读写）双收益。
- 代码现状已预留：`app/config.py:92` `region: str = "CN"`，注释明言「Phase-2 overseas amap-via-CN-proxy path」。

5. 数据层设计

5.1 表分类（23 张表全量归类）

类别	表	同步策略
参考表（共享实体，CN 生产）	pois, poi_hours, restaurants, poi_reviews, poi_stats, weather_reports	CN→海外**单向同步**；加 `_en` 列
用户表（区域私有）	users, user_profiles, entitlements, iap_transactions, paywall_rules, link_history	不同步
业务表（区域私有）	itineraries*, itinerary_days/items, intercity_transfers, execution_events, action_handoffs, trip_items, itinerary_snapshots	不同步

5.2 `_en` 列设计（延续既有模式）

现状：`pois.name_en`（models.py:55）、`restaurants.name_en`（models.py:145）**已存在**——本方案即其推广。

新增（迁移 0011，一次性）：

表	新列	译法
pois	intro_en, address_en, review_tags_en	GLM 翻译 backfill（复用 `app/services/translate.py` 快车道 + 短语缓存，§6.10）
restaurants	address_en, intro_en	同上
poi_hours/营业时间文本	hours_en（若原文为中文表述）	模板化转换（「周一至周日」→ "Mon–Sun"）优先，LLM 兜底
类目/标签枚举	不加列	代码内 i18n 字典（枚举集合有限且稳定）

语义约定：`*_en` 列**只填空、不覆盖**（同 upsert 的 fill-empty 语义）；`NULL` 时前端回退中文——保证翻译未跑完时功能完整可用。

5.3 翻译 backfill 作业（新增 `jobs/backfill_en_columns.py`）

模式照抄 `backfill_intro` / `backfill_review_summary`：分批取 `*_en IS NULL` 行 → GLM 翻译 → 只填空写回；mock LLM 下自动跳过；幂等可重跑；配 ops cmd 挂 schtasks（夜间批次，避开整点）。

5.4 参考数据单向同步（CN 导出 → 海外导入）

- 导出（CN 侧 `jobs/export_reference.py`）：参考表按 `updated_at` 水位导出 NDJSON + 清单（行数/最大 updated_at/SHA256）；weather_reports 按「未来窗口内」导出。
- 导入（海外侧 `jobs/import_reference.py`）：校验清单 → 幂等 upsert（主键/唯一键命中即整行覆盖，本地 `_en` 译文列**保留**——译文资产在海外侧重跑 backfill 补齐，避免覆盖）。
- 通道：HTTPS 对象存储中转（CN 侧对象存储 + 海外侧拉取），不做 DB 直连（跨境 DB 直连既慢又踩合规线）。
- 频率：pois/restaurants 每日一次（随 02:00 refresh 之后）；weather_reports 每日 2-3 次（随天气抓取）。
- 撤回/订正：以 CN 侧行为准——删行需墓碑标记（`deleted_at`）或清单显式删除段，防止海外残留。

6. H5 / 前端 i18n

层	方案
UI 文案	`static/i18n/{zh-CN,en}.js` 字典（key→string）；`<html lang>` + 启动语言开关切换；**不进 DB**
数据文案	API 返回中文 + `_en` 列并出（serializer 不裁剪）；H5 按 locale 取 `_en ?? 中文`
枚举/标签	前端字典映射（category、REVIEW_TAGS、错误码）
LLM 输出	规划/回复语言由请求参数 `lang` 决定，prompt 模板按语言分档（CN 版已有，EN 版 P1 落）

层	方案
部署形态	同一 H5 构建，`.env`/入口参数决定默认 locale；两域名（cn 站默认 zh，海外站默认 en），均支持手动切换

7. 区域配置矩阵（同一代码，`.env` 分档）

配置项	CN	INTL（海外）
region	CN	INTL
llm_backend / glm_base_url	智谱 open.bigmodel.cn	智谱国际端点（或本区 LLM），`lang=en` 模板
amap_base_url	restapi.amap.com 直连	经 CN 侧代理回源（config.py:91 注释既有预留）
database_url	PG-CN	PG-INTL
train_remote_enabled	True（12306 直连）	True（12306 可直连）或走代理，按实测
smtp_*	CN 邮件通道	海外邮件通道
rate_limit / share_public_*	独立调优	独立调优
state_db_url / train_cache_url	本区 SQLite	本区 SQLite

8. 实施阶段

阶段	内容	出口判据
P1（随 v24 后续迭代）	迁移 0011 加 `_en` 列；`backfill_en_columns` 作业 + ops cmd；API 并出 `_en`；H5 i18n 字典骨架 + locale 开关（CN 站内可英文预览）	CN 站切换英文全流程可用；pytest 覆盖 `_en` 并出与回退
P2（海外部署上线）	INTL `.env` 分档；高德 CN 代理；export/import 参考同步作业上线；海外 PG 迁移到 head；EN prompt 模板	海外站全流程（规划/行程/分享/导出）跑通；同步链路日级水位可见
P3（打磨）	译文质检（重点场馆/餐厅人工校订名单）；EN 站 analytics 独立；免签/预约等入境模块文案 EN 化	译文抽检达标；两站监控/告警独立值班表

9. 风险与对策

风险	对策
两套部署漂化（配置/迁移不一致）	单一 alembic 链 + 部署清单检查脚本（两侧 `alembic current` 必须同 head）；CI 双 region 配置 diff 校验
同步延迟导致海外数据陈旧	导入侧暴露 `reference_synced_at` 健康检查；滞后 >48h 告警；点评/评分类字段容忍日级滞后
译文质量（专名/菜名）	短语缓存优先（§6.10 已有）+ 重点实体人工校订名单 + `_en` 列可手改且 backfill 不覆盖（只填空语义）
跨境链路抖动（代理回源高德）	代理侧缓存 + 失败降级（直线距离/兜底时长已有 fallback 链）；INTL 侧提高 amap 超时预算

风险	对策
数据合规	用户表永不同步为硬规则（导入作业表白名单只含 §5.1 参考表）；墓碑删除贯通两侧

10. 与既有设计的关系

- 《规划引擎升级》§14 入境视角（G1 预约/G2 免签…）：这些模块天然是 EN 站核心功能，文案 EN 化在 P1/P3 落地。
- §6.10 翻译快车道：即本方案译文生产的基础设施，backfill 复用其短语缓存。
- `app/config.py` region 开关：本方案的部署开关，v1.7 预 build 时的既有预留。
- 《travel_agent_system_design V2.5》：本方案不改变单系统内部架构，仅定义第二部署与数据边界。

生成：2026-10-25 | 决策讨论：用户确认两套系统/两国部署；方案 A/B/C 对比后定稿 C+分库。实施跟踪：后续在本文追加修订记录。